

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	ATHIN SIVA.S	Reg. No.:	192524237
Section / Batch:	2025	Date:	3/8/2025

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q61. Debugging Selection Sort Code

1. Problem Interpretation

The given Selection Sort program contains a logical error. In Selection Sort, min_idx must always store the index (position) of the smallest element. However, the incorrect code stores the value of the smallest element. Later, min_idx is used as an array index, which can produce incorrect sorting or an IndexError.

2. Error Identification

Incorrect statement:

```
if arr[j] < arr[min_idx]:  
    min_idx = arr[j] # ERROR
```

Correct statement:

```
if arr[j] < arr[min_idx]:  
    min_idx = j
```

Here, j represents the position of the minimum element.

3. Corrected Selection Sort Code

```
def selection_sort(arr):  
    n = len(arr)  
  
    for i in range(n - 1):  
        min_idx = i  
  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
  
        if min_idx != i:  
            arr[i], arr[min_idx] = arr[min_idx], arr[i]  
  
    return arr  
  
arr = [64, 25, 12, 22, 11]  
  
print("Before Sorting:", arr)  
selection_sort(arr)  
print("After Sorting:", arr)
```

4. Output

Before Sorting: [64, 25, 12, 22, 11]

After Sorting: [11, 12, 22, 25, 64]

5. Step-by-Step Execution

For the array: [64, 25, 12, 22, 11]

Pass	Minimum Element	Array After Pass
1	11	[11, 25, 12, 22, 64]
2	12	[11, 12, 25, 22, 64]
3	22	[11, 12, 22, 25, 64]
4	25	[11, 12, 22, 25, 64]

Final sorted array: [11, 12, 22, 25, 64]

6. Optimized Final Code

The unnecessary self-swap is avoided using if min_idx != i:

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        min_idx = i
```

```
        for j in range(i + 1, n):
```

```
            if arr[j] < arr[min_idx]:
```

```
                min_idx = j
```

```
        if min_idx != i:
```

```
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

```
    return arr
```

7. Time Complexity

Selection Sort performs approximately $n(n - 1) / 2$ comparisons.

Best Case: $O(n^2)$

Average Case: $O(n^2)$

Worst Case: $O(n^2)$

Space Complexity: $O(1)$

Conclusion

The bug occurs because the program assigns `arr[j]` to `min_idx`, mixing up an element value with an array index. Replacing it with `min_idx = j` correctly tracks the minimum element's position. The corrected algorithm sorts the array properly while maintaining $O(n^2)$ time complexity and $O(1)$ auxiliary space.